

UPDATED PROJECT DESCRIPTION

Urdu Sentiment Analysis Using Machine Learning, Deep Learning, and Transformer Models

A detailed explanation of the improved implementation, multi-model pipeline, model evaluation, and desktop GUI

Prepared By: Sohaib Ali
Registration No: FA23-BAI-057
Program: BS Artificial Intelligence

Project Version: Improved Multi-Model Implementation

This document supersedes the earlier baseline description by explaining the newly added Logistic Regression, Linear SVM, RNN, LSTM, BERT, and XLM-RoBERTa components.

DOCUMENT PURPOSE AND STRUCTURE

Document Purpose and Structure

This report explains the improved version of the Urdu Sentiment Analysis project. The earlier project description focused mainly on text preprocessing, Bag of Words / TF-IDF features, Multinomial Naive Bayes, basic linguistic analysis, and a Tkinter user interface. The updated Python program keeps these important foundations and extends them into a complete comparative sentiment-analysis system that includes three classical machine-learning models, two deep-learning models, and two pretrained transformer models.

The purpose of this document is not only to describe what each library does, but also to show how the complete code flows from raw Urdu text to preprocessing, numerical representation, model training, testing, comparison, and GUI-based prediction.

Report Sections

1. Introduction, problem statement, and objectives
2. Dataset source and binary-class preparation
3. Improvement summary: old baseline versus improved implementation
4. Complete system workflow and module explanation
5. Text preprocessing, tokenization, word and bigram analysis
6. Feature engineering: CountVectorizer and TF-IDF with n-grams
7. Classical ML models: Naive Bayes, Logistic Regression, and Linear SVM
8. Deep learning models: RNN and LSTM
9. Transformer models: multilingual BERT and XLM-RoBERTa
10. Evaluation, model comparison, GUI operation, limitations, and future work

IMPORTANT PROJECT POSITIONING

The project is now a comparative Urdu sentiment analysis system. It trains and evaluates multiple approaches on the same cleaned binary dataset, then makes all trained approaches available through one GUI. This is a significant improvement over a single-model baseline.

1. INTRODUCTION

1. Introduction

Sentiment analysis is a Natural Language Processing (NLP) task in which a system determines the emotional orientation of text. In this project, each Urdu sentence is classified as either Positive (P) or Negative (N). For example, customer feedback, social-media comments, product reviews, public opinions, and service complaints can be processed automatically to understand whether users express satisfaction or dissatisfaction.

Urdu sentiment analysis is challenging because Urdu is written in Perso-Arabic script, words may appear with spelling variations, social-media text can contain noise, and labelled Urdu resources are much smaller than the resources usually available for English. A useful system must therefore clean noisy text, represent words numerically, learn patterns from labelled data, and evaluate predictions carefully.

Problem Statement

The initial version of the project used a traditional NLP pipeline and one main classifier. Although it demonstrated the fundamentals of Urdu sentiment analysis, a single model cannot provide a strong comparison of different learning techniques. The improved project solves this limitation by implementing and comparing classical machine learning, neural networks, and transformer-based NLP models in one codebase.

Main Objectives

- Load and validate a labelled Urdu sentiment dataset stored in TSV format.
- Remove neutral/objective samples and perform binary sentiment classification using P and N labels.
- Clean Urdu text with regular expressions and convert sentences into word tokens.

- Analyze frequent words, bigrams, simple next-word guessing, and bigram-based text generation.
- Construct count-based and TF-IDF feature spaces using unigram and bigram features.
- Train Naive Bayes, Logistic Regression, Linear SVM, RNN, LSTM, multilingual BERT, and XLM-RoBERTa approaches.
- Evaluate the models using accuracy, classification report, and confusion matrix outputs.
- Provide one Tkinter GUI that can test a custom Urdu sentence with every available model.

2. DATASET AND CLASS PREPARATION

2. Dataset and Class Preparation

The code reads a tab-separated file named `urdu-sentiment-corpus-v1.tsv`. Each record is expected to contain a text field and a label field. The project standardizes the columns as `text` and `label`, removes missing rows, removes blank text, and retains only the allowed labels P, N, and O before filtering out O.

P	Positive sentiment. The sentence communicates satisfaction, praise, happiness, approval, or another favourable opinion.
N	Negative sentiment. The sentence communicates dissatisfaction, criticism, anger, dislike, complaint, or another unfavourable opinion.
O	Objective / Neutral sentiment. The sentence does not clearly express positive or negative polarity. This class is intentionally removed by the updated implementation.

The supplied corpus paper describes a labelled Urdu Twitter corpus in which each tweet was assigned Positive, Negative, or Objective labels by multiple judges. The paper explains that the corpus is a valuable Urdu NLP resource because Urdu remains comparatively resource-poor for supervised NLP experiments. The project uses the P/N/O label convention from this type of corpus, while the actual code converts the task into a simpler and well-defined binary classification problem by removing O.

WHY NEUTRAL IS REMOVED

The code uses `df = df[df['label'] != 'O']` so that every remaining training record belongs to one of two classes. This makes the output layer, model comparison, and GUI prediction consistent: P for positive and N for negative.

Dataset Loading Function

```
def load_dataset(file_path):
    df = pd.read_csv(file_path, sep='\t', encoding='utf-8')
    df.columns = ['text', 'label']
    df.dropna(inplace=True)
    df = df[df['text'].astype(str).str.strip() != '']
    df = df[df['label'].isin(['P', 'N', 'O'])]
    df = df[df['label'] != 'O']
    df.reset_index(drop=True, inplace=True)
    return df
```

This function centralizes data validation. It prevents missing values and empty sentences from reaching the later stages. Resetting the index after filtering creates a clean continuous row index, which makes further processing and debugging easier.

3. IMPROVEMENT FROM THE EARLIER BASELINE

3. Improvement from the Earlier Baseline

The earlier description already included an important traditional NLP baseline: text cleaning, tokenization, CountVectorizer, TF-IDF, Multinomial Naive Bayes, basic word/bigram analysis, and a GUI. The new code preserves these components but adds multiple learning approaches and a formal comparison table. The improved work directly addresses the earlier report's suggested future directions of deep learning and transformer-based architectures.

Area	Earlier Description	Improved Implementation
Classification approach	Mainly Multinomial Naive Bayes	Naive Bayes + Logistic Regression + Linear SVM + RNN + LSTM + BERT + XLM-RoBERTa
Feature handling	BoW and TF-IDF explained	BoW uses count unigrams/bigrams; TF-IDF uses unigrams/bigrams for LR and SVM; sequences are prepared for RNN/LSTM
Deep learning	Suggested as future work	SimpleRNN and LSTM are trained from scratch using Keras
Transformers	Suggested as future work	Two pretrained multilingual transformer pipelines are loaded and evaluated
Comparison	Single-model performance discussion	All seven approaches are listed in one pandas comparison table
GUI	Basic interactive testing	One input sentence is tested through all available predictors in the same GUI
Linguistic analysis	Word frequency, bigram analysis, simple generation	These baseline NLP analyses are retained unchanged and integrated with the stronger modelling workflow

Therefore, the updated project is not merely a replacement of Naive Bayes with another model. It is a complete experiment framework. The same cleaned dataset is used to create appropriate input formats for different model families, each model is evaluated, and the final user interface exposes all trained predictors.

4. MODULES AND LIBRARIES USED

4. Modules and Libraries Used

The following libraries are imported in the updated code. Each one has a specific responsibility in the NLP pipeline. Together they allow data handling, text cleaning, vectorization, traditional machine learning, deep learning, transformer inference, evaluation, and user interaction.

Library / Module	Role in the Updated Project
os	Controls the TensorFlow log level through environment settings so routine execution messages do not dominate the console.
re	Uses regular expressions to remove punctuation, non-Urdu characters, and extra whitespace during preprocessing.
pandas	Reads the TSV dataset, stores it as a DataFrame, filters labels, creates cleaned text and token columns, and creates the final comparison DataFrame.
numpy	Represents numeric labels and uses argmax to convert neural-network probability outputs into P or N predictions.
Counter	Counts word frequencies and adjacent word pairs (bigrams) for linguistic exploration and simple next-word generation.
tkinter	Creates the desktop GUI where a user enters a sentence and sees predictions from the available models.
CountVectorizer	Builds a sparse count matrix using unigram and bigram features for Multinomial Naive Bayes.
TfidfVectorizer	Builds weighted unigram and bigram features for Logistic Regression and Linear SVM.
train_test_split	Creates reproducible stratified training and testing partitions with 80% training data and 20% testing data.
MultinomialNB	Classical probability-based text classifier trained on count features.
LogisticRegression	Linear discriminative classifier trained on TF-IDF features.
LinearSVC	Linear Support Vector Machine classifier trained on TF-IDF features.
TensorFlow / Keras	Builds, trains, and predicts with RNN and LSTM neural-network models.
transformers.pipeline	Loads pretrained sentiment-analysis transformer models for multilingual BERT and XLM-RoBERTa inference.
sklearn.metrics	Calculates accuracy, precision, recall, F1-score, and confusion-matrix outputs.

RUNTIME REQUIREMENT

The code must be run in an environment where the TSV dataset is available in the project folder and where the required Python packages are installed. Transformer pipelines may download model files on their first run, depending on the local cache and internet availability.

5. COMPLETE SYSTEM WORKFLOW

5. Complete System Workflow

The program is organized around a single `main()` function. The following sequence explains how data moves through the project from start to finish.

Step	Process	What Happens in the Code
1	Load data	The TSV file is read into a pandas DataFrame and invalid rows are removed.
2	Filter labels	Only valid P/N/O labels are retained; O is removed to make binary classification.
3	Clean text	Each sentence is passed through <code>preprocess()</code> to remove noise and normalize spaces.
4	Tokenize	The cleaned sentence is split into a list of words.
5	Explore text	Word frequencies, top bigrams, next-word guessing, and short bigram sentence generation are printed.
6	Create ML features	CountVectorizer and TfidfVectorizer create sparse unigram/bigram matrices.
7	Split data	The project uses a stratified 80/20 train-test split with <code>random_state=42</code> .
8	Train ML models	Naive Bayes trains on counts; Logistic Regression and Linear SVM train on TF-IDF.
9	Prepare sequences	Tokenizer converts text into word indices; <code>pad_sequences</code> makes every input length 50.
10	Train deep models	RNN and LSTM networks train on the padded sequences and numeric binary labels.
11	Load transformers	Two pretrained sentiment pipelines are initialized for BERT and XLM-RoBERTa.
12	Evaluate and compare	Accuracies are calculated, reports/matrices are printed, and a comparison DataFrame is displayed.
13	Launch GUI	The final Tkinter interface receives a sentence and requests a prediction from every model.

High-Level Pseudocode

```
load and clean TSV dataset
create tokens, word frequencies, and bigram counts
build count and TF-IDF feature matrices
split data into stratified training and testing sets
train/evaluate Naive Bayes, Logistic Regression, and Linear SVM
convert text to padded sequences; train/evaluate RNN and LSTM
load BERT and XLM-RoBERTa pipelines; evaluate them on test text
print comparison table and start GUI with all predictors
```

6. TEXT PREPROCESSING AND LINGUISTIC ANALYSIS

6. Text Preprocessing and Linguistic Analysis

6.1 Preprocessing Function

Raw text can contain punctuation, quotation marks, numbers, English characters, extra spacing, or symbols that are not useful for a word-based Urdu classifier. The `preprocess()` function removes these sources of noise before any vectorization or neural sequence conversion.

```
def preprocess(text):
    text = str(text)
    text = re.sub(r"[punctuation marks]", ' ', text)
    text = re.sub(r'[^\u0600-\u06FF\s]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

The first substitution replaces punctuation with a space so that two adjacent words do not merge after the punctuation is removed. The second substitution keeps only characters from the Urdu Unicode range and whitespace. The third substitution converts repeated spaces into one space and removes spaces from the beginning and end of the sentence. The output is a cleaner, more consistent representation of the original text.

6.2 Tokenization

Tokenization divides a cleaned sentence into individual word units. The code uses the simple operation `text.split()`. This is appropriate for a basic word-level sentiment-analysis pipeline because later components such as `CountVectorizer`, `TF-IDF`, and `Keras Tokenizer` require a sequence or collection of words.

6.3 Word Frequency Analysis

After tokenization, the code flattens all token lists into one sequence and sends it to `Counter`. The output `word_freq.most_common(10)` reports the ten most frequent words. This analysis is useful for exploratory data analysis because it shows which terms dominate the training corpus. Very frequent function words may have little sentiment value, whereas words strongly associated with praise or criticism may become meaningful model features.

6.4 Bigram Analysis, Next-Word Guessing, and Generation

A bigram is a pair of consecutive tokens. The `build_bigram_counts()` function visits each token list and counts pairs such as `word[i]` followed by `word[i+1]`. Bigrams are important because a sentiment is often expressed by phrases rather than by a single word. For example, a positive or negative adjective can receive stronger meaning when it occurs with its neighbouring modifier.

The function `guess_next_word()` finds all bigrams whose first word matches a given word. It selects the most common second word as the best simple prediction. The function `generate_sentence()` repeats this process for a fixed number of steps. These two functions do not replace a real language model; they are educational demonstrations of how statistical word-pair frequencies can support basic next-word prediction and short text generation.

WHY THESE ANALYSES ARE RETAINED

Although the improved project includes advanced transformer models, the word and bigram functions remain valuable. They demonstrate core NLP concepts, expose corpus patterns, and make the project easier to explain in a viva or presentation.

7. FEATURE ENGINEERING FOR TRADITIONAL MODELS

7. Feature Engineering for Traditional Models

Traditional machine-learning models cannot directly process raw text. Therefore, the project converts each cleaned Urdu sentence into a numeric vector. The new code uses two different vectorizers because different models benefit from different numerical representations.

7.1 CountVectorizer for Naive Bayes

`CountVectorizer` creates a Bag-of-Words style sparse matrix. Each column corresponds to a vocabulary feature and each row corresponds to one sentence. The number in a cell represents how many times that feature appears in that sentence. The project sets `ngram_range=(1, 2)`, so the vocabulary includes both single words (unigrams) and two-word phrases (bigrams).

```
vectorizer = CountVectorizer(
    tokenizer=lambda x: x.split(),
    ngram_range=(1, 2)
)
X_counts = vectorizer.fit_transform(df['clean_text'])
```

This representation is a natural fit for Multinomial Naive Bayes because that classifier models the distribution of discrete feature counts across classes. Bigrams allow the model to recognize local phrase patterns that may not be visible from single words alone.

7.2 TF-IDF for Logistic Regression and Linear SVM

TF-IDF means Term Frequency - Inverse Document Frequency. It still reflects whether a term appears in a document, but it gives more weight to terms that are informative and less common across the overall corpus. Common words that occur everywhere receive lower importance, while discriminative terms can receive higher importance.

```
tfidf_vectorizer = TfidfVectorizer(
    tokenizer=lambda x: x.split(),
    ngram_range=(1, 2)
)
X_tfidf = tfidf_vectorizer.fit_transform(df['clean_text'])
```

The updated code trains Logistic Regression and Linear SVM on TF-IDF because both are strong linear text-classification baselines when the input is a high-dimensional sparse feature matrix. The program also prints the total number of CountVectorizer and TF-IDF features so the user can inspect the size of the learned vocabularies.

7.3 Reproducible, Stratified Train-Test Split

The program uses `test_size=0.2`, which means 80% of the data is used for training and 20% for testing. The parameter `random_state=42` makes the split reproducible. The parameter `stratify=df['label']` preserves the positive/negative class proportion in both sets, which produces a more reliable comparison.

8. CLASSICAL MACHINE LEARNING MODELS

8. Classical Machine Learning Models

The improved code trains three classical machine-learning models. These models are computationally efficient, widely used in text classification, and useful as baselines against which deep-learning and transformer approaches can be compared.

Model	Input Representation	Working Principle in This Project
Multinomial Naive Bayes	CountVectorizer unigrams + bigrams	Estimates how likely each feature count is under P and N classes, then selects the class with the stronger posterior probability.
Logistic Regression	TF-IDF unigrams + bigrams	Learns a linear decision boundary that estimates the probability of a positive versus negative label from weighted features.
Linear SVM	TF-IDF unigrams + bigrams	Learns a maximum-margin separating boundary between positive and negative feature vectors.

8.1 Multinomial Naive Bayes

Multinomial Naive Bayes is a probability-based classifier designed for discrete count data. It assumes that the contribution of each feature is conditionally independent given the class. This assumption is simplified, but the model remains effective and very fast for many text-classification tasks. In the code, it is trained with the CountVectorizer output.

```
nb_model = MultinomialNB()
nb_model.fit(X_train_counts, y_train)
acc_nb = evaluate_model('Naive Bayes', nb_model, X_test_counts, y_test)
```

8.2 Logistic Regression

Logistic Regression is a linear discriminative classifier. For binary sentiment analysis, it learns a weight for every TF-IDF feature and combines those weights to estimate class membership. A positive score pushes the prediction toward P, while a negative score pushes the prediction toward N. The parameter `max_iter=1000` gives the solver enough iterations to converge on high-dimensional text features.

```
lr_model = LogisticRegression(max_iter=1000, solver='liblinear')
lr_model.fit(X_train_tfidf, y_train)
acc_lr = evaluate_model('Logistic Regression', lr_model, X_test_tfidf, y_test)
```

8.3 Linear Support Vector Machine

Linear SVM finds a separating hyperplane between positive and negative TF-IDF vectors. It focuses on maximizing the margin, which is the distance between the decision boundary and the closest training examples from each class. For sparse text features, linear SVM is commonly a strong baseline. The code sets `max_iter=10000` to allow a sufficiently long optimization process.

```
svm_model = LinearSVC(max_iter=10000)
svm_model.fit(X_train_tfidf, y_train)
acc_svm = evaluate_model('Linear SVM', svm_model, X_test_tfidf, y_test)
```

9. DEEP LEARNING MODELS: RNN AND LSTM

9. Deep Learning Models: RNN and LSTM

Deep-learning models require a different input format from classical vectorizers. Instead of a sparse matrix of term counts or TF-IDF weights, neural networks receive ordered sequences of word indices. The Keras Tokenizer learns a vocabulary from the cleaned training corpus, converts words into integer IDs, and reserves an out-of-vocabulary token for unseen words.

9.1 Sequence Preparation

```
tokenizer_keras = Tokenizer(num_words=10000, oov_token='<OOV>')
tokenizer_keras.fit_on_texts(texts)
sequences = tokenizer_keras.texts_to_sequences(texts)
max_len = 50
X_sequences = pad_sequences(sequences, maxlen=max_len,
                           padding='post', truncating='post')
```

The project limits the vocabulary to 10,000 words. Each text is converted to a list of token IDs. Because sentences have different lengths, `pad_sequences()` makes every sequence exactly 50 positions long. Short sentences receive zeros at the end, while longer sentences are truncated after 50 tokens. This fixed shape is required for efficient neural-network training.

9.2 Label Encoding

The original labels are strings: N and P. The deep-learning models use numeric labels, so the code maps N to 0 and P to 1. A NumPy array stores these numeric labels. The final neural layer has two outputs and uses softmax activation to produce one probability for negative and one for positive.

9.3 RNN Architecture

The SimpleRNN model processes the sequence from one word position to the next. Its hidden state carries information from earlier words, allowing the network to consider word order. The architecture in this project uses an embedding layer, a 64-unit SimpleRNN layer, dropout, a 32-unit ReLU dense layer, and a two-class softmax output layer.

```
Embedding(vocab_size, 64, input_length=max_len)
SimpleRNN(64)
Dropout(0.4)
Dense(32, activation='relu')
Dense(2, activation='softmax')
```

The embedding layer converts each integer word ID into a learned 64-dimensional dense vector. The RNN receives these vectors in sequence, learns patterns that correlate with sentiment, and returns a representation that is passed to the dense classifier. Dropout randomly disables part of the learned representation during training to reduce overfitting.

9.4 LSTM Architecture

The LSTM model uses the same general structure as the RNN model, but replaces SimpleRNN with LSTM(64). LSTM uses gated memory cells, enabling it to preserve or forget information more selectively across a word sequence. This makes it more suitable than a simple RNN for dependencies that span several words.

Both neural models are compiled with sparse categorical cross-entropy loss and the Adam optimizer. They are trained for five epochs with batch size 32 and a 10% validation split from the training data. The program predicts class probabilities for test sequences, applies `np.argmax()`, and calculates accuracy, a classification report, and a confusion matrix.

INTERPRETING RNN VS LSTM

RNN and LSTM scores must be compared after running the same train-test split. LSTM may capture longer patterns more effectively, but actual performance depends on dataset size, vocabulary quality, class balance, hyperparameters, and random initialization.

10. TRANSFORMER MODELS: BERT AND XLM-ROBERTA

10. Transformer Models: BERT and XLM-RoBERTa

The updated project additionally uses two pretrained transformer pipelines through Hugging Face Transformers. Unlike models trained from scratch on this small local dataset, pretrained transformers already contain language knowledge learned from large multilingual text collections. The project uses them as ready-made sentiment classifiers and tests their predictions on the same held-out Urdu text used for evaluation.

10.1 Multilingual BERT Sentiment Pipeline

The first pipeline loads the model `nlptown/bert-base-multilingual-uncased-sentiment`. Its original output represents star ratings. The code converts that output into binary labels by mapping ratings containing 1 or 2 to N, ratings containing 4 or 5 to P, and applying N as the fallback for other labels. In practice, this means the middle rating is treated as negative because the project only has two target classes.

```
bert_pipe = pipeline('sentiment-analysis',
    model='nlptown/bert-base-multilingual-uncased-sentiment',
    tokenizer='nlptown/bert-base-multilingual-uncased-sentiment',
    device=-1)
```

10.2 XLM-RoBERTa Sentiment Pipeline

The second pipeline loads `cardiffnlp/twitter-xlm-roberta-base-sentiment`. Its output labels are mapped by checking whether the returned label contains the word positive. If it does, the code returns P; otherwise it returns N. Because the project is binary, non-positive output is grouped into negative.

```
xlmr_pipe = pipeline('sentiment-analysis',
    model='cardiffnlp/twitter-xlm-roberta-base-sentiment',
    tokenizer='cardiffnlp/twitter-xlm-roberta-base-sentiment',
    device=-1)
```

10.3 Transformer Evaluation Method

The code keeps raw cleaned test texts in `raw_test_texts`. It applies the BERT and XLM-RoBERTa predictor functions one text at a time, then compares the returned P/N labels with `raw_test_labels` using accuracy. This produces transformer scores that can be placed alongside the scores of the supervised classical and deep models.

ACADEMIC NOTE ABOUT LABEL MAPPING

The transformer models are pretrained for their own sentiment label schemes, not specifically fine-tuned on this exact Urdu corpus. The P/N mapping is a practical integration rule in the code. For a stronger research version, validate the mapping carefully or fine-tune the transformer on the project dataset.

11. EVALUATION AND MODEL COMPARISON

11. Evaluation and Model Comparison

11.1 Evaluation Metrics

The function `evaluate_model()` accepts a model, test feature matrix, and true labels. It predicts labels and prints three main evaluation outputs: accuracy, classification report, and confusion matrix.

Metric	Meaning in the Project
Accuracy	The proportion of test sentences classified correctly. It provides an overall result but should not be the only metric considered.
Precision	For a predicted class, precision measures how many predicted examples actually belong to that class.
Recall	For a true class, recall measures how many actual examples of that class the model successfully detects.
F1-score	The harmonic balance of precision and recall. It is useful when a model has uneven performance across positive and negative classes.
Confusion Matrix	A 2 x 2 table that reveals correct positive, correct negative, false positive, and false negative predictions.

11.2 Comparison Table Generated by the Code

After all models are evaluated, the program creates a pandas DataFrame with the model name and dynamically calculated accuracy. This table provides one clear location to compare seven approaches. The code prints the DataFrame in the console before opening the GUI.

```
comparison = [
    {'Model Name': 'Naive Bayes', 'Accuracy': f'{acc_nb:.4f}'},
    {'Model Name': 'Logistic Regression', 'Accuracy': f'{acc_lr:.4f}'},
    {'Model Name': 'Linear SVM', 'Accuracy': f'{acc_svm:.4f}'},
    {'Model Name': 'RNN', 'Accuracy': f'{acc_rnn:.4f}'},
    {'Model Name': 'LSTM', 'Accuracy': f'{acc_lstm:.4f}'},
    {'Model Name': 'BERT', 'Accuracy': f'{acc_bert:.4f}'},
    {'Model Name': 'XLM-RoBERTa', 'Accuracy': f'{acc_xlmr:.4f}'},
]
```

RESULTS TO INSERT AFTER EXECUTION

The uploaded source code calculates scores dynamically, but no console-output log was provided with it. Therefore, this report does not invent numerical accuracies. Before submission, run the program successfully, copy the real accuracy values and confusion matrices from the console, and add them to the results section below.

Model	Actual Accuracy	Observation to Write After Running
Naive Bayes	Insert runtime value	Baseline performance using count unigrams and bigrams.
Logistic Regression	Insert runtime value	Linear TF-IDF classifier; compare with Naive Bayes.
Linear SVM	Insert runtime value	Maximum-margin TF-IDF classifier; compare with Logistic Regression.
RNN	Insert runtime value	Sequence model with SimpleRNN memory.
LSTM	Insert runtime value	Gated sequence model; compare with RNN.
BERT	Insert runtime value	Pretrained multilingual star-rating model mapped to binary labels.
XLM-RoBERTa	Insert runtime value	Pretrained multilingual Twitter-sentiment model mapped to binary labels.

12. GUI-BASED MULTI-MODEL PREDICTION

12. GUI-Based Multi-Model Prediction

The Tkinter interface converts the project from a console-only experiment into a usable desktop application. The GUI window contains a prompt, one text input field, one Predict Sentiment button, and an output label. The user enters an Urdu sentence and clicks the button. The program then sends the sentence to each predictor in the predictors dictionary.

The dictionary connects a model name to a small function that performs the correct preprocessing and prediction route for that model. Classical models transform text with CountVectorizer or TF-IDF; deep models transform it into padded token sequences; transformer models call their pretrained pipeline. The GUI gathers all returned P/N values and prints one result line per model.

```

predictors = {
    'Naive Bayes': lambda text: predict_nb(...),
    'Logistic Regression': lambda text: predict_lr(...),
    'SVM': lambda text: predict_svm(...),
    'RNN': lambda text: predict_rnn(...),
    'LSTM': lambda text: predict_lstm(...),
    'BERT': lambda text: predict_bert(...),
    'XLM-RoBERTa': lambda text: predict_xlmr(...),
}

```

Why the GUI is a Strong Improvement

- It makes testing easy for a non-technical user because the user does not need to call functions manually.
- It displays the agreement or disagreement among the seven models on the same sentence.
- It demonstrates practical deployment of NLP models rather than only offline training.
- It allows quick testing of new Urdu text after the models have been trained.

Rule-Based Helper Function

The source code also includes a function named `predict_sentiment()` that checks a small list of strong positive and negative phrases before falling back to a vectorizer/model prediction. This helper illustrates a hybrid rule-based plus machine-learning idea. However, the final predictors dictionary currently calls `predict_nb()` for Naive Bayes instead of this rule-based helper. The function remains available for future integration or experiments.

13. LIMITATIONS, RECOMMENDATIONS, AND FUTURE WORK

13. Limitations, Recommendations, and Future Work

13.1 Current Limitations

- The code removes neutral/objective sentences, so it cannot predict a separate Neutral class.
- The RNN and LSTM models use a limited vocabulary, a fixed sequence length, and only five training epochs; their performance may improve or decrease with hyperparameter changes.
- The transformer models are pretrained external models and are not fine-tuned on the exact project dataset.
- Transformer label conversion is simplified to P/N. In particular, BERT middle/star outputs and non-positive XLM-RoBERTa outputs are mapped to N.
- The application trains all local models every time the script starts. Saving trained models and vectorizers would reduce startup time in a production version.
- The GUI reports raw P/N labels only. A future UI can present full words, confidence scores, color indicators, and saved prediction history.

13.2 Future Enhancements

- Keep a three-class setup: Positive, Negative, and Neutral, rather than discarding O.
- Use Urdu-specific normalization, stemming/lemmatization, stop-word handling, and spelling-variation handling.
- Fine-tune multilingual BERT or XLM-RoBERTa directly on the Urdu P/N dataset using a validation set and hyperparameter tuning.
- Add model persistence using joblib for classical models and Keras save/load functions for neural networks.
- Show confidence values, majority voting, and charts in the GUI.
- Use cross-validation for classical models and repeat neural experiments with multiple random seeds.
- Expand the dataset with non-political domains such as product reviews, restaurant feedback, education, and e-commerce comments.

14. CONCLUSION

14. Conclusion

The updated Urdu Sentiment Analysis project demonstrates a complete multi-model NLP framework. It begins with a labelled Urdu corpus, removes invalid and neutral records, cleans Urdu script, tokenizes text, performs word and bigram analysis, and prepares three types of model inputs: count vectors, TF-IDF vectors, and padded word sequences.

The project then trains and evaluates Multinomial Naive Bayes, Logistic Regression, Linear SVM, SimpleRNN, and LSTM models. It also integrates two pretrained transformer sentiment pipelines: multilingual BERT and XLM-RoBERTa. Every approach is brought together through one comparison table and a Tkinter GUI, where users can test the sentiment of a new Urdu sentence with all models.

Compared with the earlier single-model baseline, this updated version is academically stronger because it demonstrates traditional machine learning, deep learning, transformer NLP, comparative evaluation, and practical GUI deployment in one coherent project. The final result is not just a sentiment classifier; it is a structured Urdu NLP experiment that can be extended through better preprocessing, fine-tuning, saved models, and improved dataset coverage.

15. REFERENCES AND SOURCE MATERIALS

15. References and Source Materials

[1] Updated implementation source: Pasted code.py supplied by Sohaib Ali. The code contains the complete multi-model Urdu sentiment analysis script, including preprocessing, vectorizers, ML models, RNN/LSTM, transformer pipelines, evaluation, and GUI.

[2] Previous project report: Project Description (057) supplied by Sohaib Ali. It describes the earlier Urdu Sentiment Analysis version based mainly on preprocessing, Bag of Words, TF-IDF, Multinomial Naive Bayes, word/bigram analysis, and Tkinter GUI.

[3] Khan, Muhammad Yaseen and Nizami, Muhammad Suffian. Urdu Sentiment Corpus (v1.0): Linguistic Exploration and Visualization of Labeled Dataset for Urdu Sentiment Analysis. Provided corpus-paper PDF; it documents Urdu tweet data with Positive, Negative, and Objective labels and discusses data collection, cleaning, and labelling.

FINAL SUBMISSION CHECKLIST

Run the code with the TSV dataset in the correct project folder; capture the actual seven model accuracies and confusion matrices; replace the placeholder values in Section 11.2; then submit this description together with the final working source code.